

Machine Intelligence:: Deep Learning

Beate Sick, Lilach Goren, Pascal Bühler

Institut für Datenanalyse und Prozessdesign
Zürcher Hochschule für Angewandte Wissenschaften

Outline of the DL Module (tentative)

The course is split in 8 sessions, each 4 lectures long. Topics might be adapted during the course

Day	Date	Time	Topic
1	15.04.2025	09:00-12:30	Introduction to Deep Learning & Keras, first NNs
-	21.04.2025	-	FRÜHLINGS-FERIEN
-	28.04.2025	-	FRÜHLINGS-FERIEN
2	06.05.2025	09:00-12:30	Loss, Optimization, Regression, Classification
3	13.05.2025	09:00-12:30	Computer vision, CNN-architecture
4	20.05.2025	09:00-12:30	DL in practice, pretrained (foundation) models
5	27.05.2025	09:00-12:30	Model evaluation, baselines, xAI, troubleshooting
6	03.06.2025	09:00-12:30	Generative Models, Transformer-architecture
7	10.06.2025	09:00-12:30	Vision Transformer
8	17.06.2025	09:00-12:30	Projects, deep Ensembling

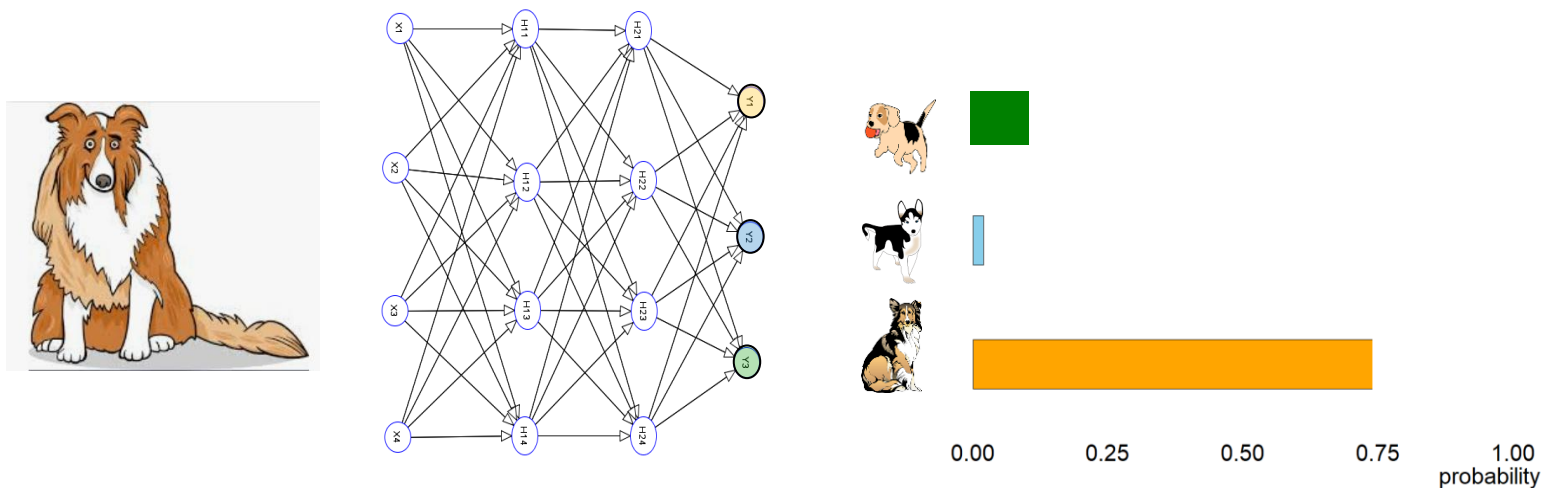


Ensemble models

- Improve prediction performance (test AUC, NLL)
- Improve uncertainty quantification

What do we get from a DL classification model?

Suppose you train a classifier to discriminate between 3 dog breeds

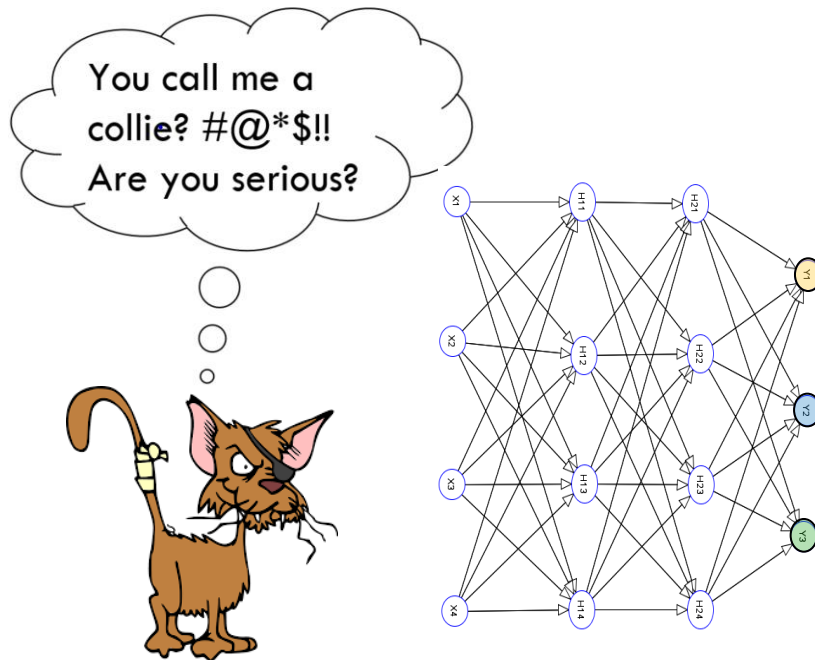


The prediction is “collie” because it gets the highest probability: $p_{\max}=0.75$

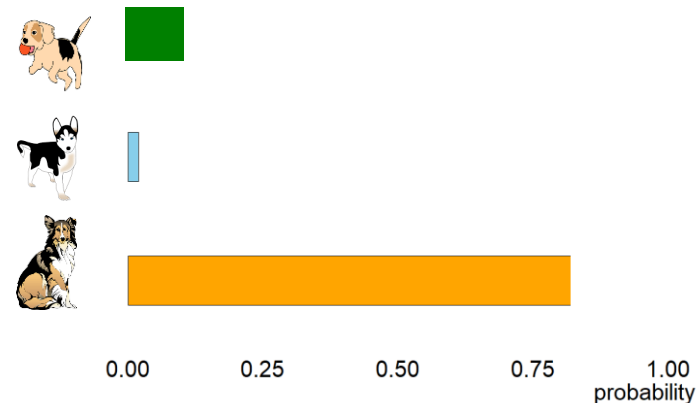
What is 0.75 telling us?

In a calibrated classifier in 75 out of 100 predictions we have a Collie

What happens if a novel class is presented to the CNN?

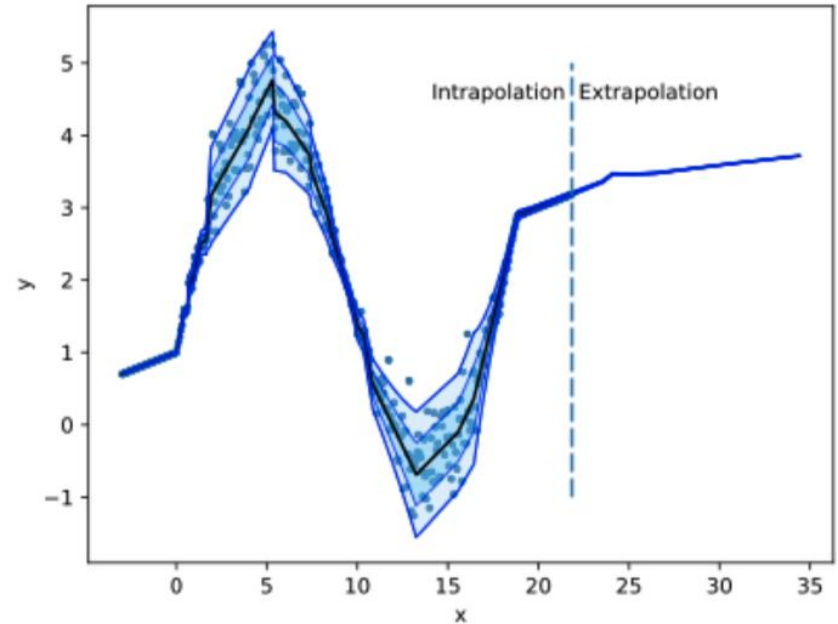
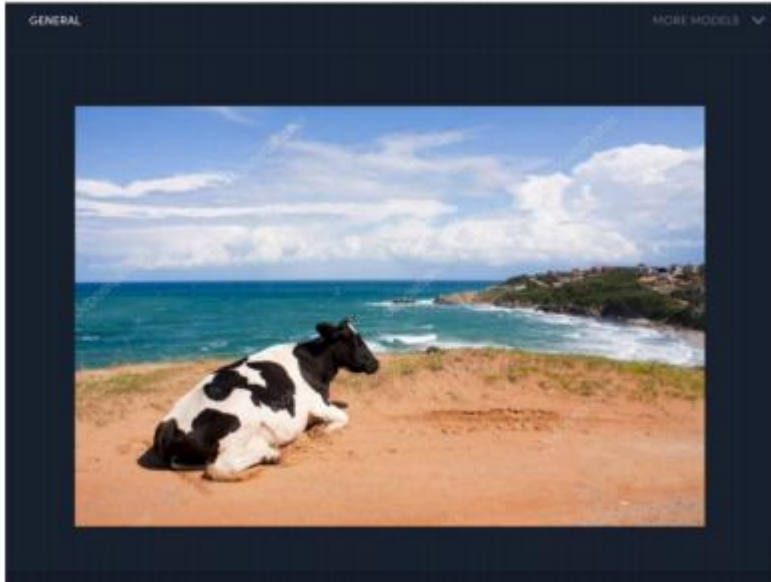


Plain wrong !



The reported probability is only valid if, we have the $P(\text{Train})=P(\text{Test})$. That's not the case. "The big lie deep learning is based on". This is more evident, if we have a look at regression problems.

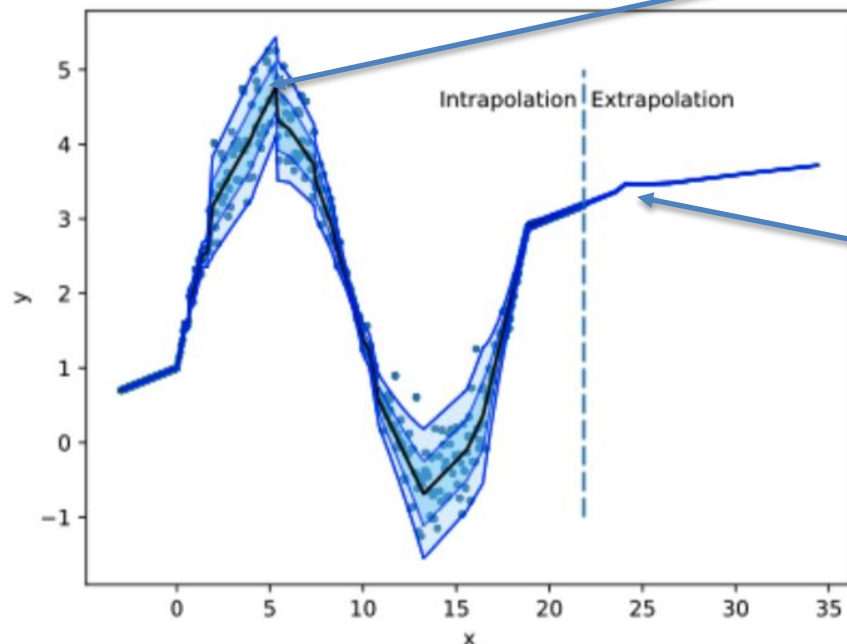
Extrapolation



Deep Learning is not robust against out of distribution data (OOD)

Aleatronic vs. Epistemic Uncertainty

Much spread in data, aleatronic (from “Alea Acta est”))



No epistemic uncertainty, in standard DL

- *Aleatoric* uncertainty is due to the uncertainty in the data (also in the limit of $N \rightarrow \infty$)
- The uncertainty when leaving the ‘known ground’ is called *epistemic* uncertainty.

We can model the epistemic uncertainty when we take the uncertainty with which we know the weights (called parameter uncertainty) into account.

DL should know when it doesn't



- In many situations, we can't guaranty $P(\text{Train})=P(\text{Test})$.
- DL better state this uncertainty

Deep Ensembling

Original paper by Lakshminarayanan et al.: <https://arxiv.org/pdf/1612.01474.pdf>

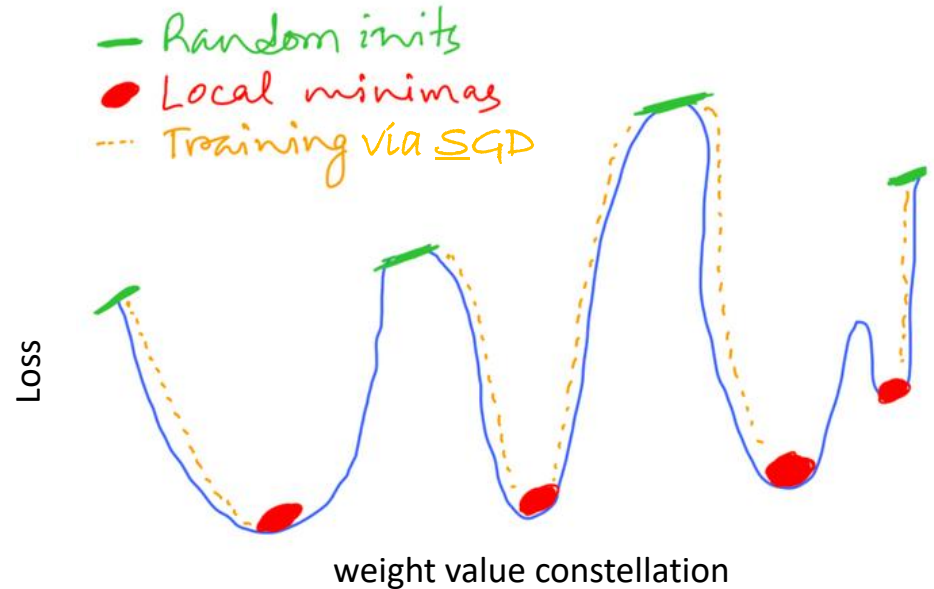
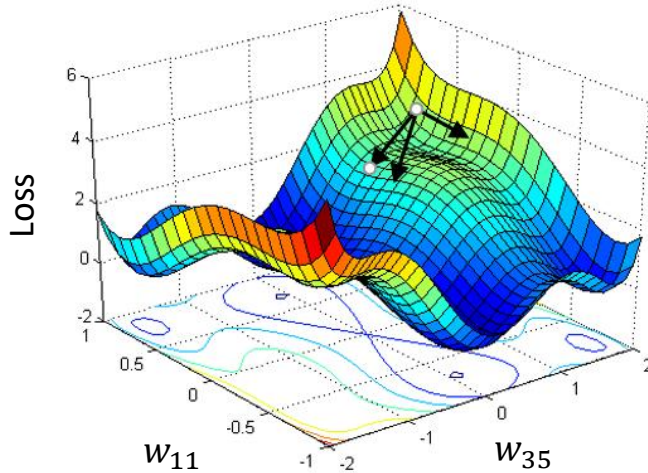
Basic Idea of deep ensembling

- Use an ensemble of n -different models and see if they agree
- How to get different ensemble members?
- Traditional Statics?
 - Bagging Boot Strapping and Aggregating
- Deep Learning
 - Deep Learning just average over different solutions of gradient descent

Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems*, 30.

The loss-landscape in DL is usually not convex

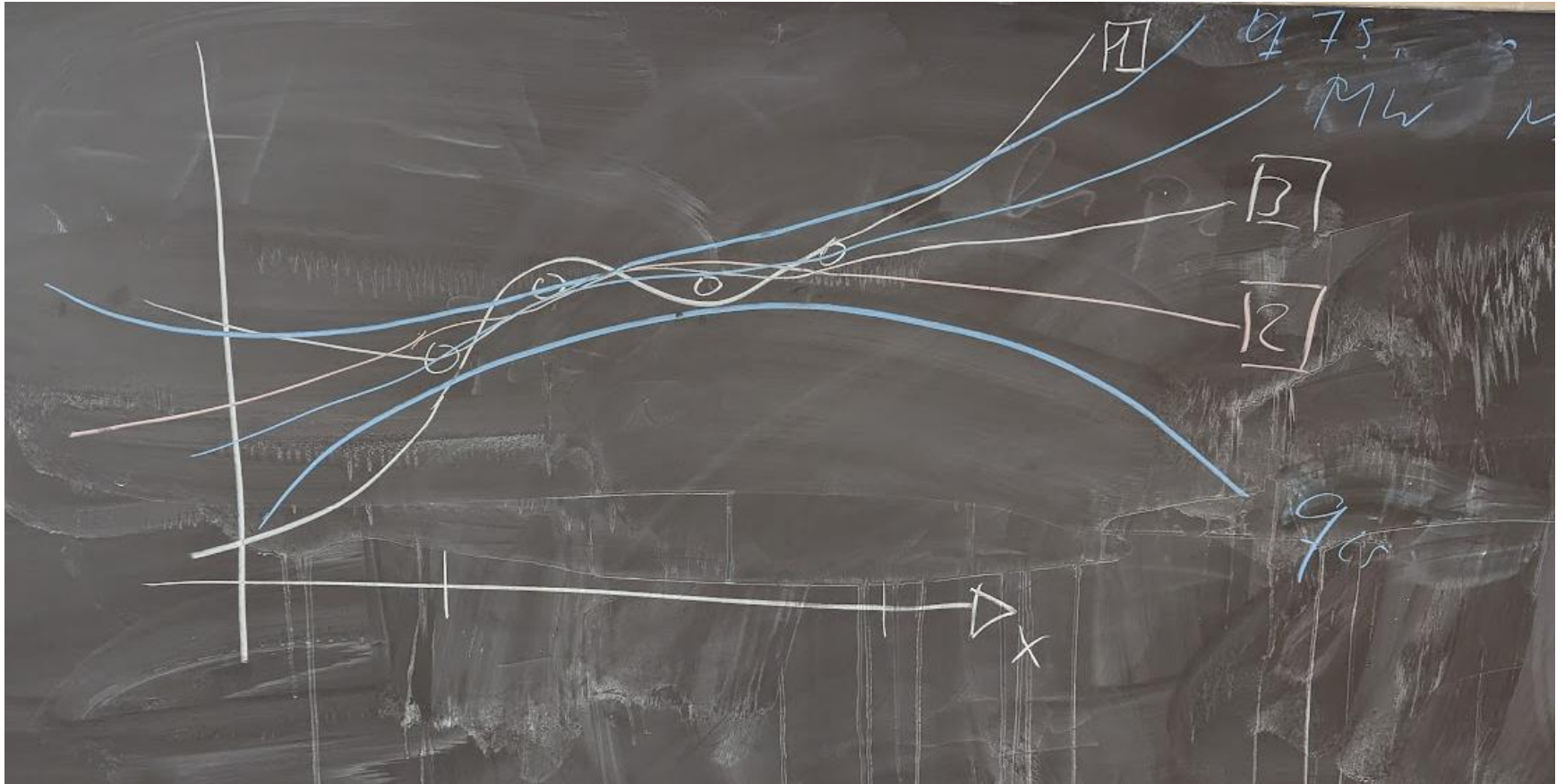
Loss-landscape



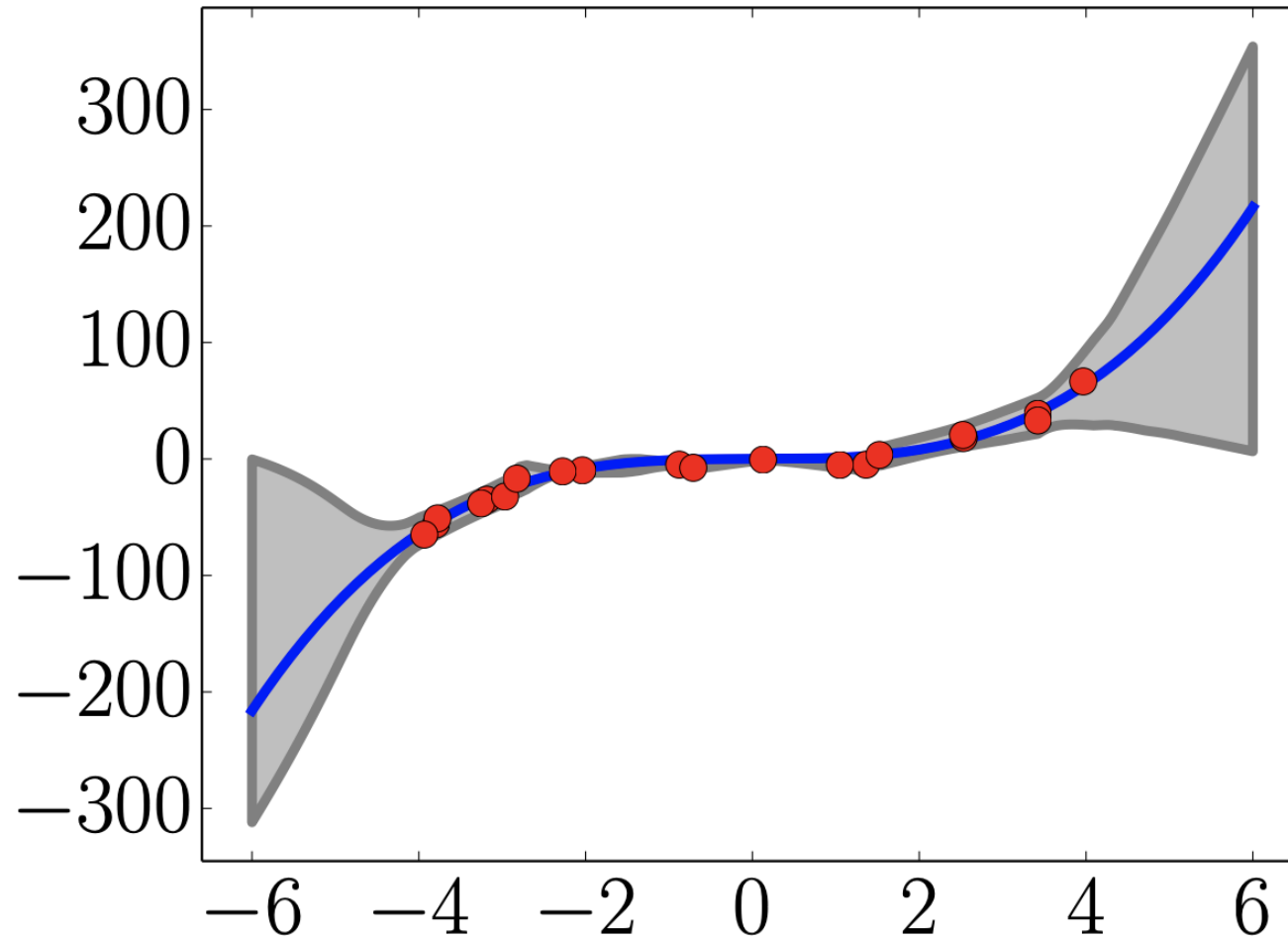
The loss-landscape of DL models has many local minima with similar depth.

Training is started with a **random** weight value initialization → training the NN with **SGD** and the same data several times is usually ending in different local minima.

Blackboard Regression Ensembling with 3 solutions



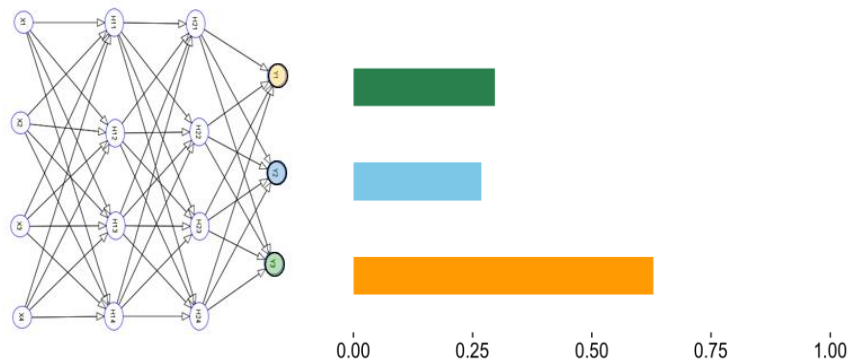
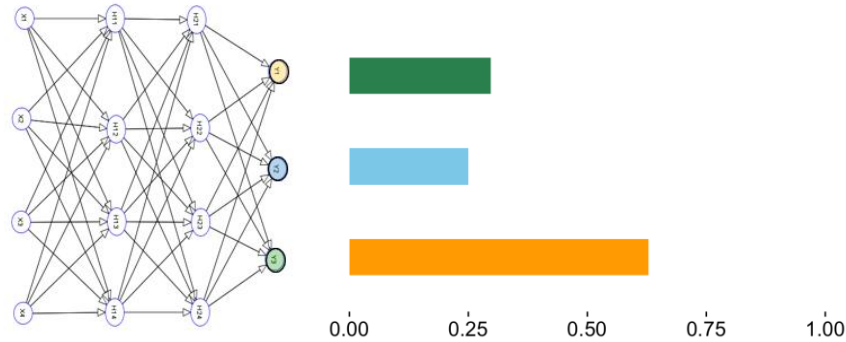
Ensemble of 5 Networks (Regression)



Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems*, 30.

Classification

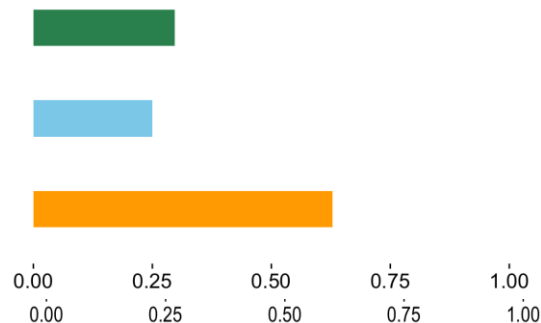
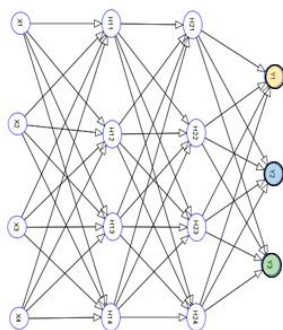
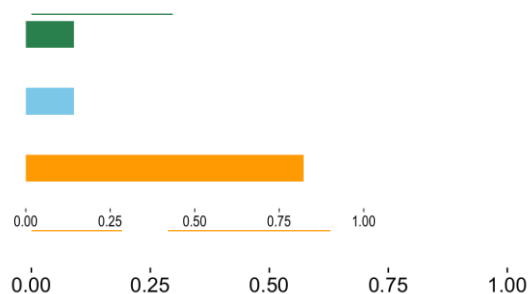
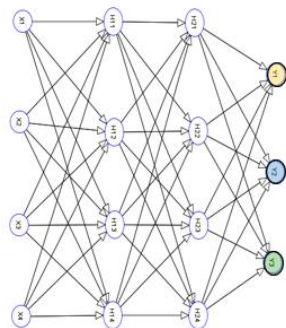
What happens if we have trained the same CNN twice with the same data? Present example in training set.



Small difference if example is know

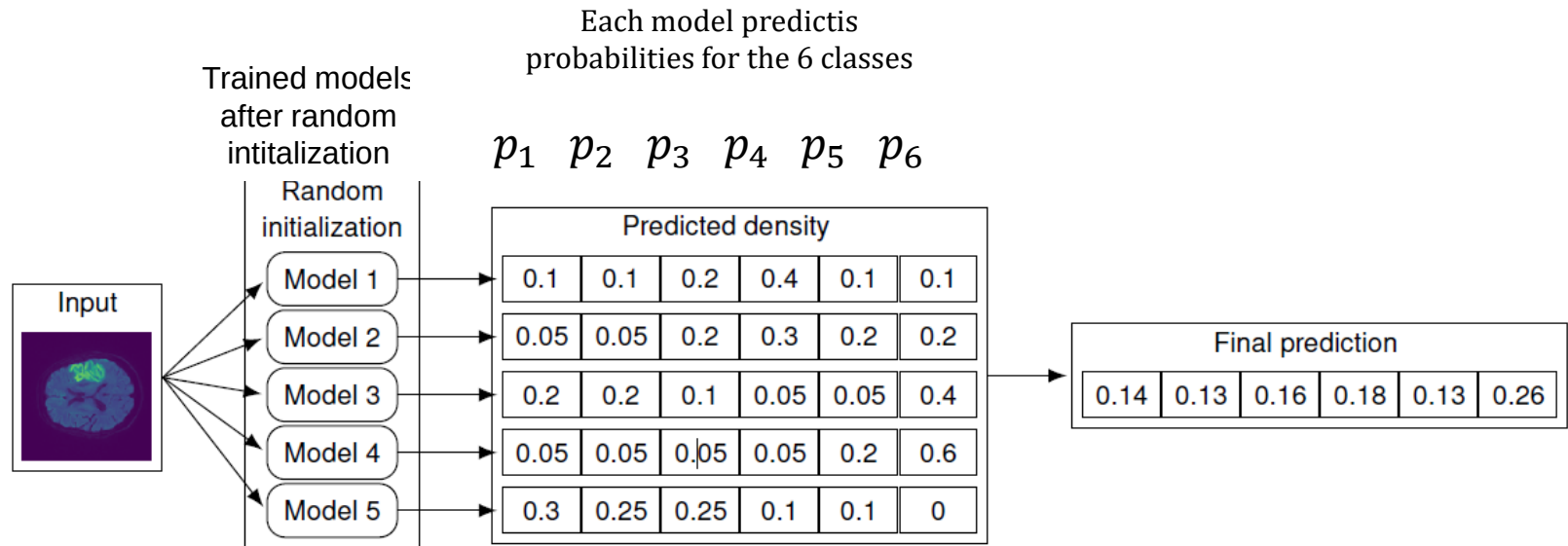
Classification

What happens if we have trained the same CNN twice with the same data? But present OOD example.



Larger difference if not (cat).

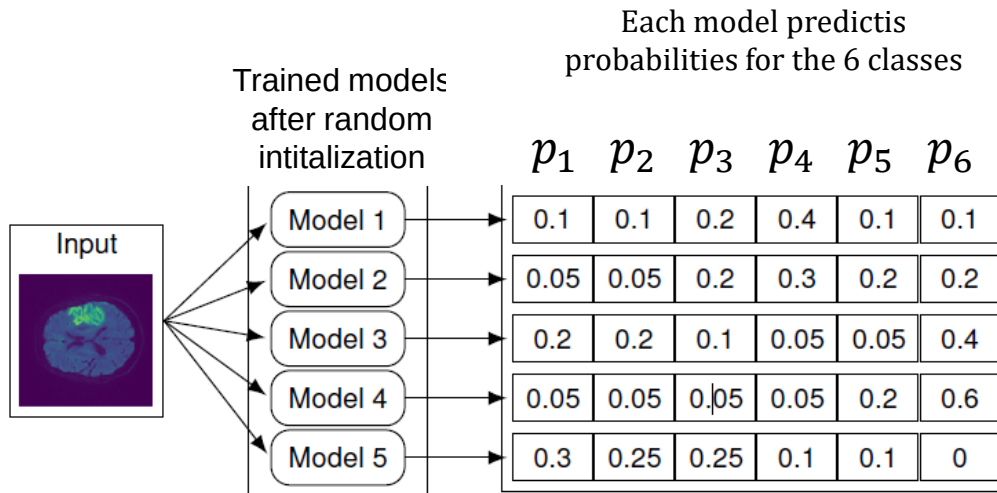
Deep ensembling: Train several NN models and average their predictions



We get the **ensemble predictions** by averaging the probabilities that were predicted by the different models for each class

Added benefit. Ensembling also has better classification performance (e.g. NLL), then single average of the individual models.

Deep ensembling: Train several NN models and average their predictions



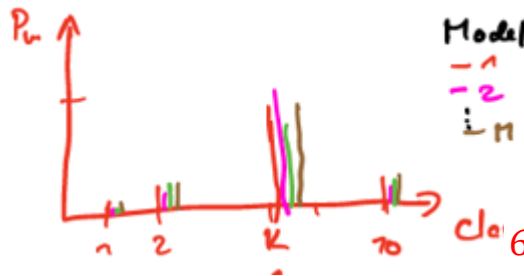
We get the **ensemble predictions** by averaging the probabilities that were predicted by the different models for each class

p_1^E	p_2^E	p_3^E	p_4^E	p_5^E	p_6^E
0.14	0.13	0.16	0.18	0.13	0.26

We can get the **aleatoric ensemble uncertainties** by computing the entropy of the ensemble prediction $H(P^E) = -\sum_i p_i^E \cdot \log(p_i^E)$.

We can get the **epistemic ensemble uncertainties** by e.g. computing the standard deviation of the probabilities that were predicted by the different models for each class

$$sd_1^E \quad sd_2^E \quad sd_3^E \quad sd_4^E \quad sd_5^E \quad sd_6^E$$



Nice:

For the NLL loss, it is guaranteed, that the NLL of the ensemble prediction is better (smaller or equal) than the average NLL of the individual models.

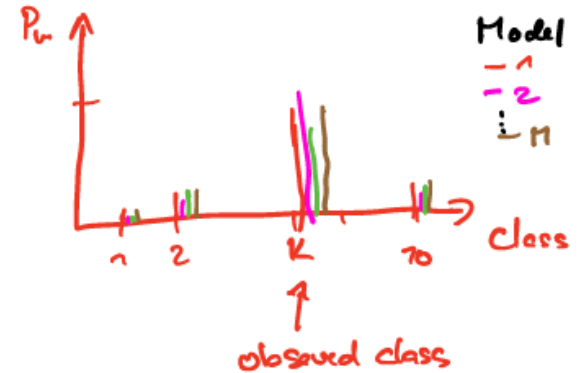
Ensembling improves the NLL performance (Just for Reference)

Ensemble prediction for an observation
with observed class = k , based on M models:

$$P_k^E = \frac{1}{M} \sum_{m=1}^M P_{km} \quad k = 1 \dots 10 \text{ for 10 classes}$$

associated NLL contribution l :

Model m : $l_m = -\log P_{km}$ / Ensemble: $l^E = -\log P_k^E$



to show $\bar{l} \geq l^E$

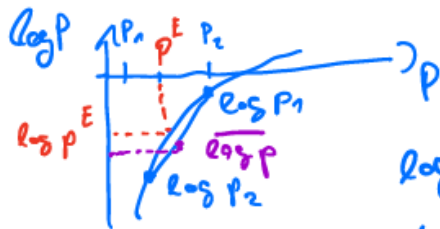
$$\text{with } \bar{l} = \frac{1}{M} \sum_{m=1}^M l_m = \frac{1}{M} \sum_{m=1}^M -\log P_{km}$$

$$= - \frac{1}{M} \sum_{m=1}^M \log P_{km}$$

$$\leq \log \frac{1}{M} \sum_{m=1}^M P_{km}$$

$$\geq -\log P_k^E = l^E \quad \square$$

use Jensen inequality



$$\log \bar{p} \geq \overline{\log p}$$

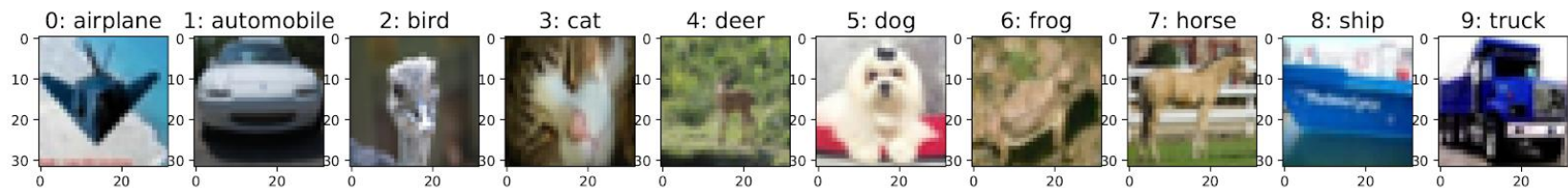
$$\log \frac{1}{M} \sum_{m=1}^M P_m \geq \frac{1}{M} \sum_{m=1}^M \log P_m$$

Hands-on Time



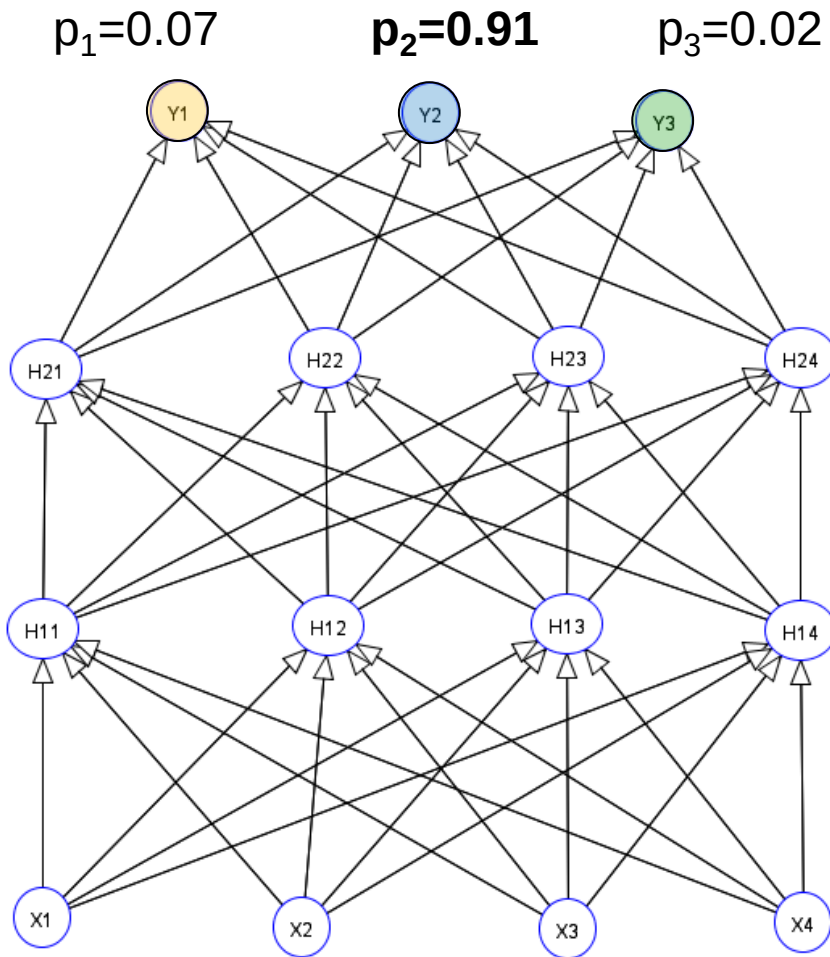
Notebook deep ensembling

Train 5 times CNN on cifar 10 data for constructing an ensemble model and show that the prediction performance improves



**MC Dropout during test time
as ensembling w/o retraining**

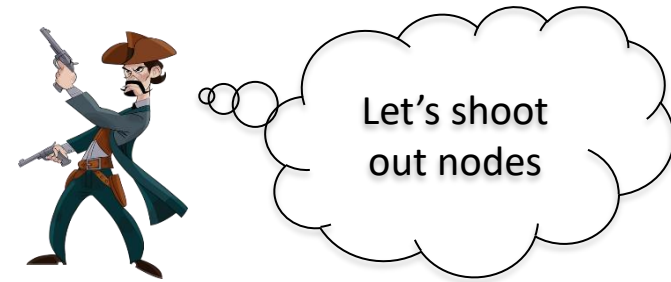
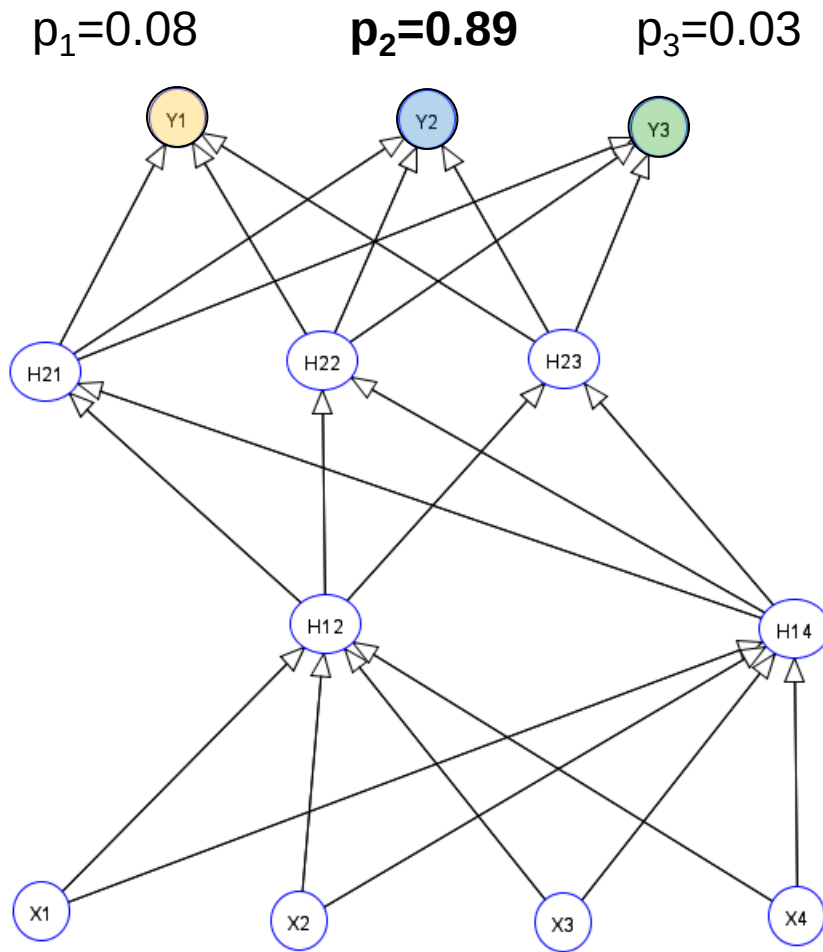
No MC Dropout



Probability of predicted class: $p_{\max}$

Input: image pixel values

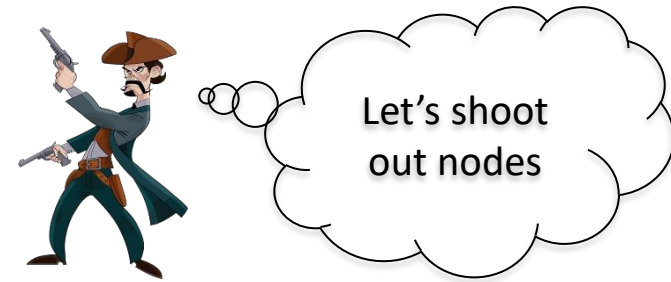
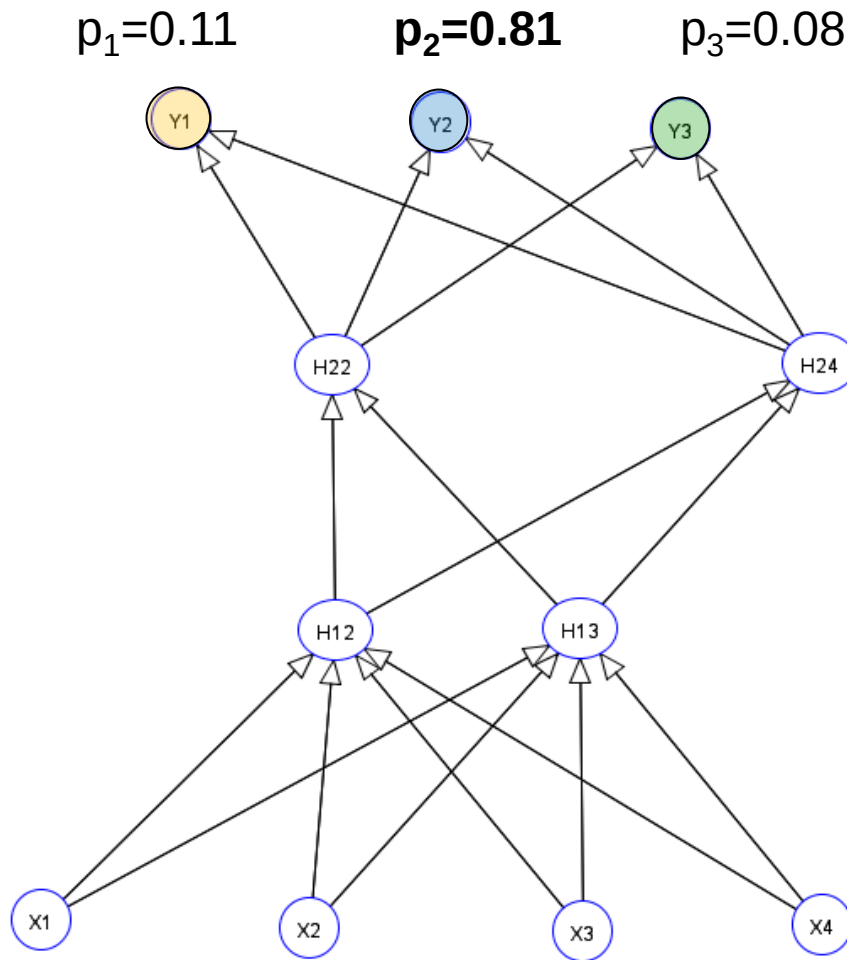
Use dropout at test time: Run 1



Stochastic dropout of units

Same input image

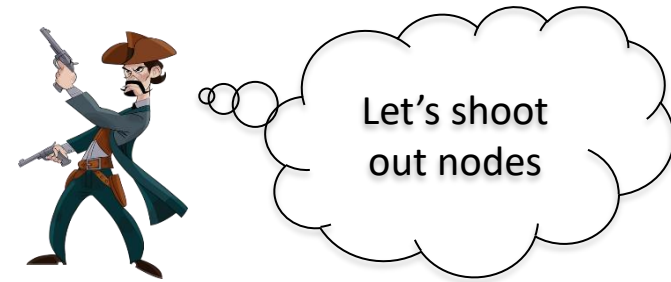
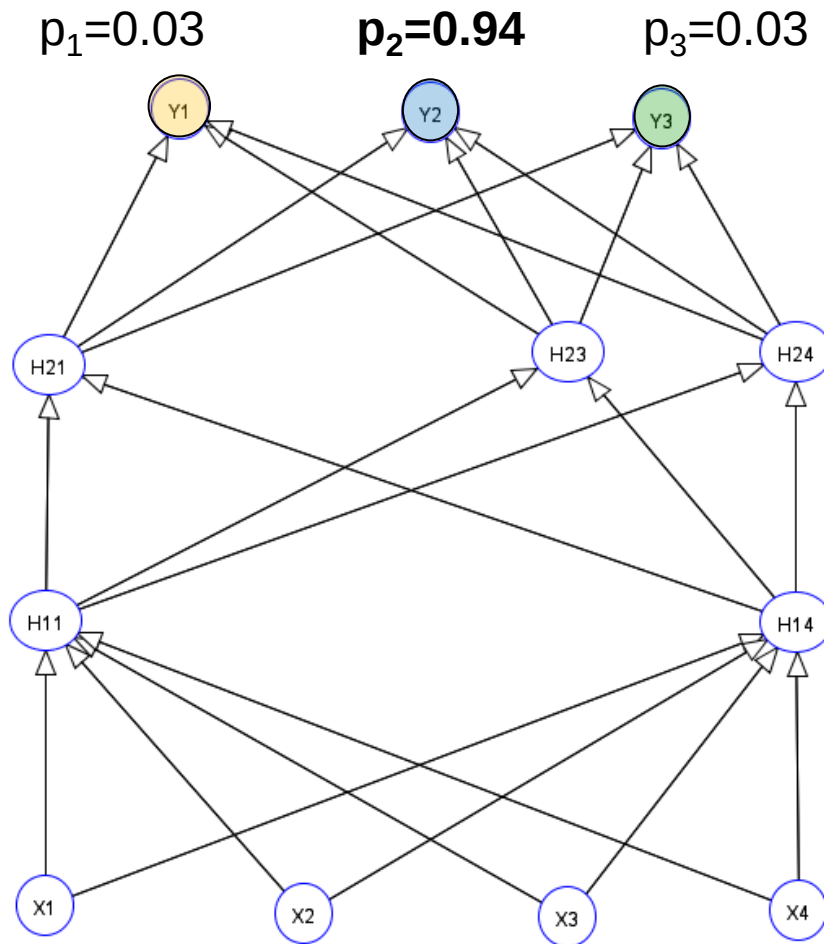
Use dropout at test time: Run 2



Stochastic dropout of units

Same input image

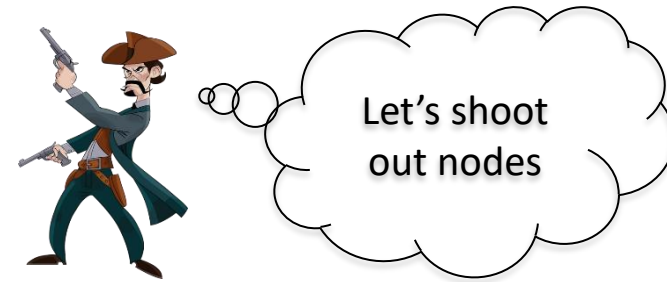
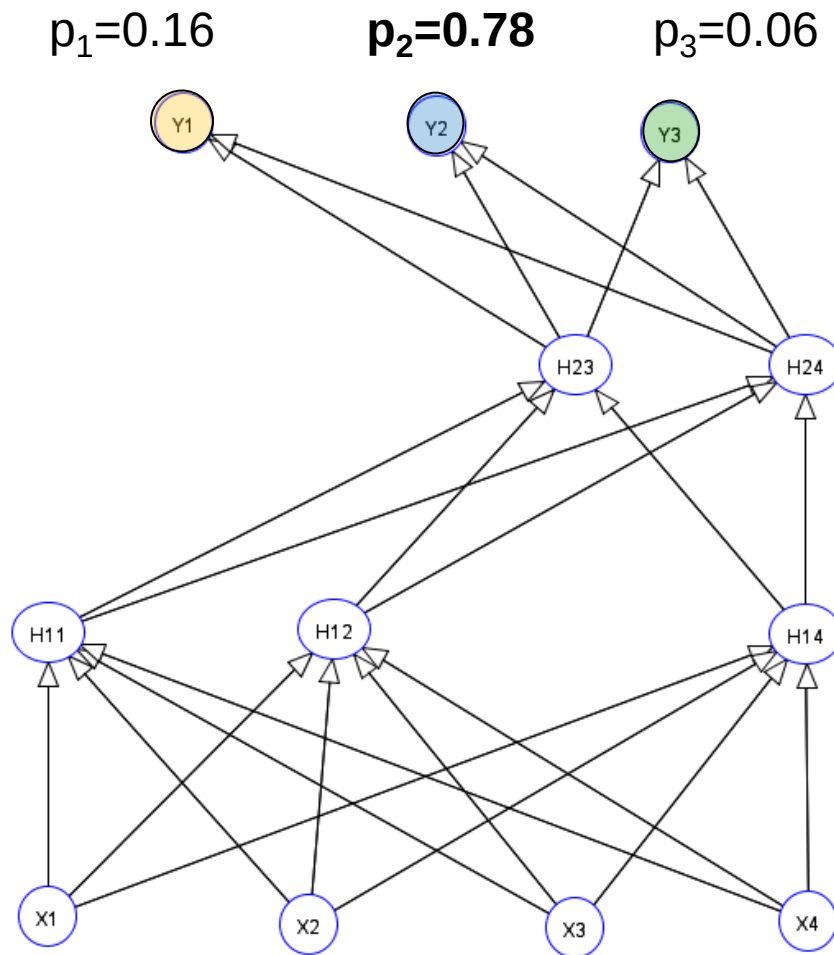
Use dropout at test time: Run 3



Stochastic dropout of units

Same input image

Use dropout at test time: Run 4



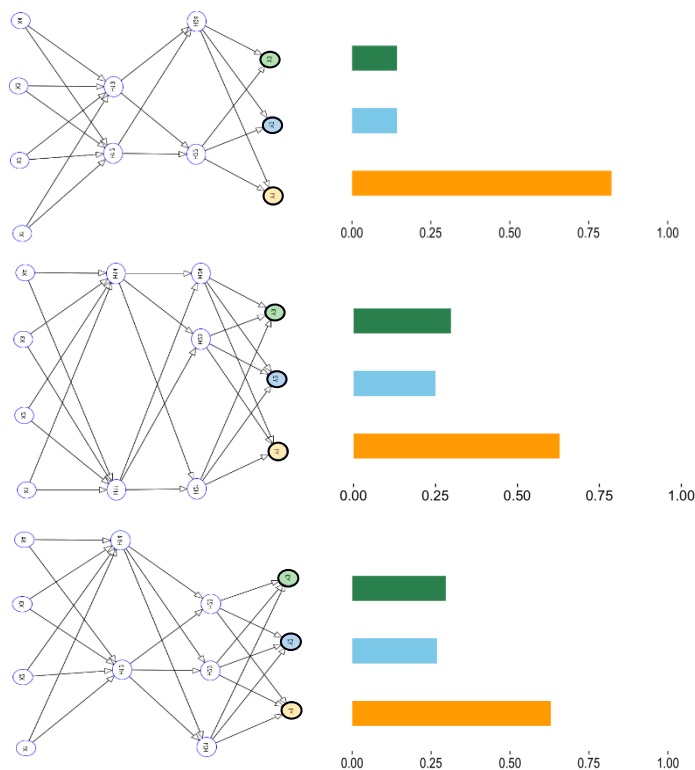
Stochastic dropout of units

Same input image

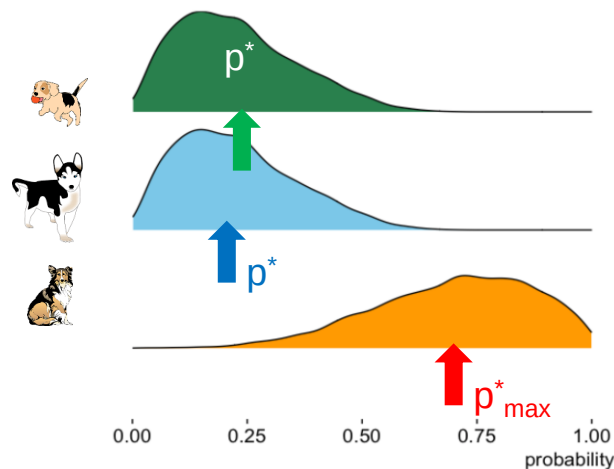
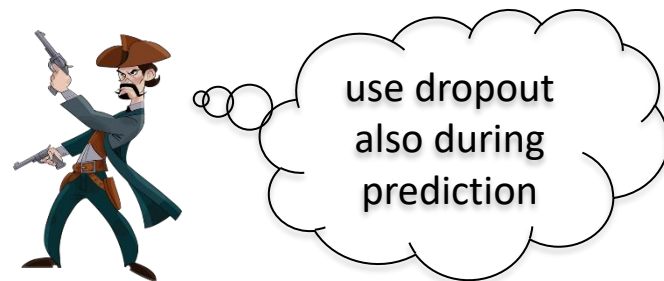
MC probability prediction



Many Dropout Runs in forward pass



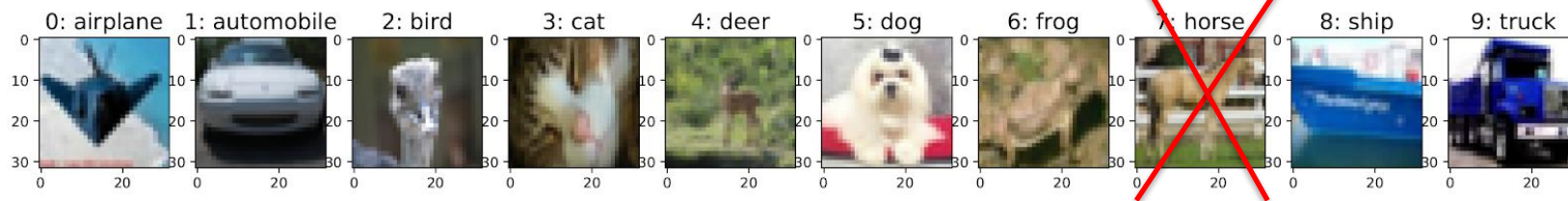
...



MC-dropout prediction is the average p^* of the probability distribution
Uncertainty is given by the standard deviation of the distribution

Remark: Mean of marginal give components of mean in multivariate distribution.

Hands-on Time cntd.: Fit the VI Bayesian NN

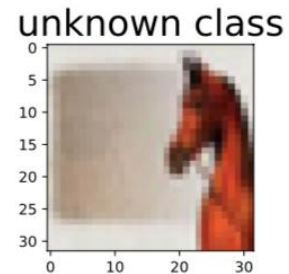
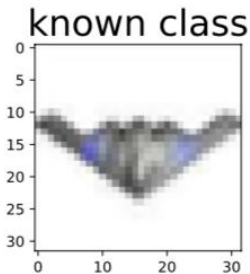


Train a CNN with only 9 of the 10 classes and investigate if the uncertainties are different when predicting images from known or unknown classes.

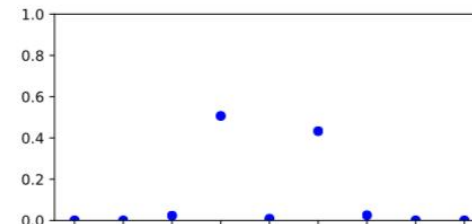
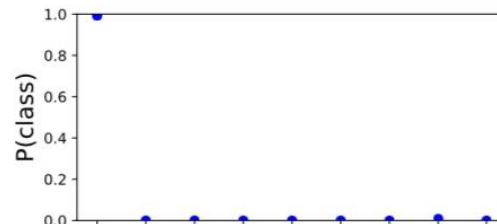
https://github.com/tensorchiefs/dl_course_2021/blob/master/notebooks/20_cifar10_classification_mc_and_vi.ipynb

Looking at the predictive distribution!

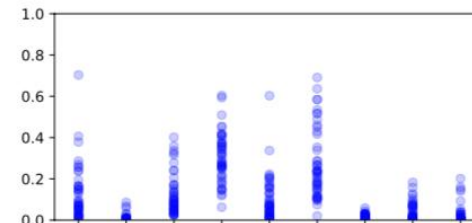
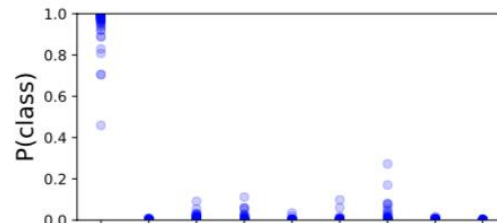
Input image



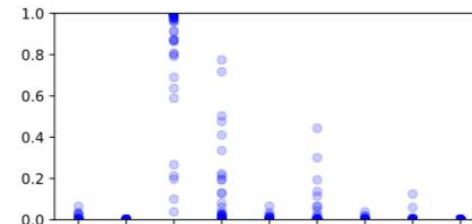
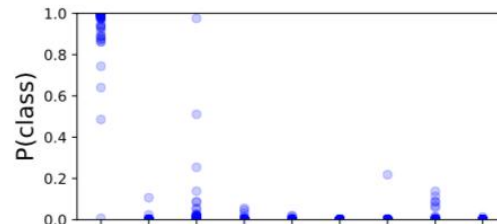
Non-Bayesian CNN



Bayesian CNN via VI



Bayesian CNN via dropout



Summary

- Model averaging aka ensemble models yield better prediction performance
 - Ensemble models allow to assess part of the epistemic uncertainty useful for novel class detection or alarm rising in case of wrong predictions
-
- Deep Ensembling NNs
 - Train a NN $m=5$ times with random initialization on same data
 - Average the predicted probability distributions of the 5 trained NNs
 - Pro: very simple approach capturing the algorithmic part of the epistemic uncertainty
 - Con: computational expensive - requires to train a NN several times
 - MC Dropout during test time
 - Perform MC Dropout not only during training but also after training when using NN
 - Average the predicted probability distributions of the different MC dropout NNs
 - Pro: very simple & computationally cheap (only 1 NN needs to be trained)
 - Con: performs in practice often worse than deep ensembling